

Aula 13 — Fundamentos ReactJS

Módulo: ReactJS + .NET

Carga horária: 4 horas

Projeto base: Frontend React (inicialmente mockado)

Objetivo da aula:

Compreender os **fundamentos do ReactJS**, o conceito de **SPA**, o uso de **JSX**, **componentes**, **props** e **state**, e construir o **primeiro CRUD mockado**, preparando a base para integração com **Web API .NET**.

PARTE 1 — TEORIA (≈ 2 horas)

SPA — Single Page Application

◆ O que é uma SPA?

Aplicação web que:

- Carrega **uma única página HTML**
- Atualiza a UI dinamicamente via JavaScript
- Comunica-se com backend via **API**

Fluxo:

Browser → index.html → React renderiza componentes → API (.NET) fornece dados (JSON)

◆ Vantagens

- ✓ UX fluida
- ✓ Separação Front / Back
- ✓ Escalabilidade

◆ Desvantagens

- ✗ SEO mais complexo
 - ✗ Segurança depende do backend
-

2 JSX (JavaScript XML)

◆ O que é?

Sintaxe que mistura **HTML** + **JavaScript**.

Exemplo:

```
const titulo = "Produtos"; return <h1>{titulo}</h1>;
```

📌 Observações importantes:

- JSX não é HTML puro
 - Usa `className` em vez de `class`
 - Expressões JS ficam entre `{ }`
-

3 Componentes

◆ Conceito

React é baseado em **componentes reutilizáveis**.

Tipos:

- **Componentes de função** (padrão atual)
- Componentes de classe (legado)

Exemplo:

```
function Header() { return <h1>Meu Sistema</h1>; }
```

- ✓ Reutilização
 - ✓ Manutenção
 - ✓ Organização
-

4 Props e State

◆ Props

- Dados passados do pai para o filho
- Somente leitura

```
function ProdutoItem({ nome }) { return <li>{nome}</li>; }
```

◆ State

- Estado interno do componente
- Quando muda → re-render

```
const [contador, setContador] = useState(0);
```

📌 Regra de ouro:

Props descem, eventos sobem.

PARTE 2 — PRÁTICA (≈ 2 horas)

Objetivo Prático

Configurar um projeto React moderno e criar um **CRUD mockado de Produtos**, utilizando **componentes, props, state e JSX**.

1 Setup do Projeto React

✓ Criar projeto com Vite (padrão atual)

```
npm create vite@latest aula13-react -- --template react cd aula13-react npm install npm run dev
```

Estrutura:

```
src |— components |— App.jsx |— main.jsx
```

2 Limpando o Projeto Base

 App.jsx

```
import Produtos from "../components/Produtos"; function App() { return ( <div> <h1>React + .NET</h1> <Produtos /> </div> ); }  
export default App;
```

3 Criando o Componente Produtos (State)

 components/Produtos.jsx

```
import { useState } from "react"; import ProdutoForm from "../ProdutoForm"; import ProdutoList from "../ProdutoList"; function  
Produtos() { // Estado principal da aplicação (mockado) const [produtos, setProdutos] = useState([ { id: 1, nome: "Notebook",  
preco: 4500 }, { id: 2, nome: "Mouse", preco: 150 } ]); function adicionarProduto(produto) { setProdutos([...produtos, produto]);
```

```
} function removerProduto(id) { setProdutos(produtos.filter(p => p.id !== id)); } return ( <div> <ProdutoForm onAdicionar={adicionarProduto} /> <ProdutoList produtos={produtos} onRemover={removerProduto} /> </div> ); } export default Produtos;
```

- ✓ State central
 - ✓ Lógica no componente pai
-

4 Formulário (Props + State local)

components/ProdutoForm.jsx

```
import { useState } from "react"; function ProdutoForm({ onAdicionar }) { const [nome, setNome] = useState(""); const [preco, setPreco] = useState(""); function submit(e) { e.preventDefault(); const novoProduto = { id: Date.now(), nome, preco }; onAdicionar(novoProduto); setNome(""); setPreco(""); } return ( <form onSubmit={submit}> <input placeholder="Nome" value={nome} onChange={e => setNome(e.target.value)} /> <input placeholder="Preço" value={preco} onChange={e => setPreco(e.target.value)} /> <button type="submit">Adicionar</button> </form> ); } export default ProdutoForm;
```

- ✓ Controlled Components
 - ✓ Evento sobe via props
-

5 Lista de Produtos (Props)

components/ProdutoList.jsx

```
function ProdutoList({ produtos, onRemover }) { return ( <ul> {produtos.map(produto => ( <li key={produto.id}> {produto.nome} - R$ {produto.preco} <button onClick={() => onRemover(produto.id)}> Remover </button> </li> ))} </ul> ); } export default ProdutoList;
```

- ✓ Renderização com map
- ✓ key obrigatório
- ✓ Comunicação pai ⇌ filho

Conceitos Aplicados (Resumo)

Conceito	Onde foi usado
SPA	React renderizando UI
JSX	Todos os componentes
Componentes	Produtos, Form, List
Props	Comunicação
State	Dados e UI

Resultado Esperado

- ✓ Projeto React configurado
 - ✓ CRUD mockado funcional
 - ✓ Compreensão clara de props/state
 - ✓ Base sólida para consumo de API .NET
-

Atividade Avaliativa

Desafio prático:

1. Adicionar campo `categoria`
2. Criar componente `Header`
3. Exibir total de produtos
4. Impedir cadastro com nome vazio

5. Estilizar minimamente (CSS simples)